

Adapting Tool Documentation for Cross-Hop Parameter Passing

Sergey Bugaev
ITMO University, Saint Petersburg

Advisor: Stanislav Batalenkov

14 September 2026

Abstract

Language agents can be asked to solve multi-hop questions by calling REST APIs, given only the vendor catalog as documentation. Those catalogs are written for humans: they name the next endpoint and leave implicit which previous call yields a path identifier and which fields in a list payload are stubs. The model then invents an id, the call is rejected, and the trace still looks well-formed. We study which documentation slot of a released depth-first controller should carry that binding. We compile fill and response contracts from the catalog and a handful of live probes, and write them into those slots. A fill contract names the producer call and the JSON path of each path parameter; a response contract names the returned fields and the hops those identifiers feed. The object is the slot, not a learned rewriter. On RestBench-TMDB, replicated on Ling and DeepSeek V4, a fill line in the guideline is more executable than a free-form purpose rewrite on both families. Against the original catalog it wins on Ling and matches on DeepSeek V4, while the free-form rewrite fails on both.

1 Introduction

To answer a multi-hop REST question (retrieve a collection, read the movie list, fetch a review), the agent is given only the vendor catalog as documentation [10, 7]. Toolformer, ReAct, and Gorilla already show that a language model can emit a search query, a calculator call, or an API name [9, 13, 6]. Pointed at that catalog, the same idea still depends on what the catalog says.

That catalog is written for a human reader. Each entry names a purpose and a URL template. It does not say which earlier call produces the identifier that belongs in the template, or which keys in a list payload are real fields rather than stubs. The model is left to invent the binding. On a live backend the invention is cheap to detect: the path parameter is missing or wrong, HTTP rejects the call, and the trace can still contain every gold API name.

Existing edits of the same documents rarely target that binding as a fill slot. DRAFT rewrites the purpose paragraph from trial feedback [8]; EasyTool compresses a vendor page into a short instruction [14]. Both change the purpose slot; the guideline read after the hop is chosen stays a human sentence. Trace-Free+ learns a description rewriter and, on RestBench, scores it under teacher forcing [1]. Arazzo and OpenAPI Links already write a producer path as a workflow spec [5]. AutoRestTest and APIPilot recover producer–consumer edges to generate tests, not to fill an existing DFSDT catalog [2, 3]. Gorilla, Toolformer, ReAct, and ToolGen change the model, the index, or the prompting loop [6, 9, 13, 12]. The write we study is a typed producer path in the fill slot of a catalog that already exists.

Figure 1 shows that binding failure on one RestBench-TMDB query. The user asks for a review of a movie from a named collection. Search finds the collection, but the review endpoint still needs a movie identifier. Under the vendor catalog the collection id is filled and the movie id is left as a stub: the live call is rejected and there is no review. Under DRAFT both stubs stay empty. Under the compiled fill contract the controller writes the correct id from the collection parts list, and HTTP returns a user review.

Give me a review of a movie
from the collection The Fast and the Furious.

Catalog	DRAFT	Ours
GET /search/collection → 9485	GET /search/collection	GET /search/collection → 9485
GET /collection/9485	GET /collection/collection_id	GET /collection/9485
GET /movie/movie_id/reviews	GET /movie/movie_id/reviews	GET /movie/584/reviews
✗ missing path parameter movie_id ✗ No review.	✗ missing path parameter movie_id ✗ No review.	✓ 200 2 Fast 2 Furious ✓ 6/10 — John Chard.

Figure 1: The same RestBench-TMDB query under three documents. Catalog and DRAFT emit the review hop and leave the movie id unbound; the live call is rejected and there is no review. The fill contract writes the correct id from the collection parts list and returns a user review.

We compile two short contracts from the catalog and a handful of live probes, without gold paths. A *fill* contract on every hop API names the producer calls and the JSON path of each path parameter. A *response* contract on every search or list API names the returned fields and the downstream hops those identifiers feed. Both go into the documentation slots of a released depth-first controller [7], leaving the search procedure and the request schema as they are.

The edit is a typed line in the fill slot, not a rewrite of the purpose sentence. Fill is kept out of the selection surface of hop APIs, because extra text there makes the agent repeat search and skip the id call; response scope is compressed on catalog tools only. The same sentence in another slot is a different intervention.

The rest of the paper evaluates that slot write on RestBench against the vendor catalog and against DRAFT, on a live TMDB backend and two model families. The tables, like Figure 1, score whether the last hop executes. Agreement with gold API names can stay high in every arm. Where the same payload helps one family and blocks another, that limit belongs to the edit, not to a new generator of descriptions.

2 Related work

REST agents

RestGPT connects a language model to live vendor REST APIs and releases RestBench: multi-hop questions on TMDB and Spotify, scored as an ordered path against gold relevant APIs [10]. We keep those queries and that gold list. A gold name sequence can still fail HTTP, so we also score whether every call on the path is accepted by the live backend. ToolLLM’s DFSDT controller searches a catalog depth-first [7]. `choose_tool` reads a purpose u_t ; `choose_parameter` reads a guideline d_t . We use the released driver and change only the strings in those slots.

Models, indexes, and prompting loops

Toolformer trains a language model to insert API calls into its own generations [9]. Gorilla finetunes a model to write API calls and retrieves documentation at inference [6]. ReAct interleaves a reasoning trace with an action [13]. ToolGen represents each of tens of thousands of APIs as a vocabulary token and trains retrieval and calling as next-token prediction on ToolBench [12]. Intra-collection splits already require one call’s output to become another call’s input; the training target is still the API name, and the catalog strings stay as found. Each of these changes how the model is trained, indexed, or prompted.

Purpose-slot rewrites

DRAFT lets a model explore an API and rewrites the purpose paragraph from trial feedback [8]. EasyTool compresses a vendor page into a short instruction [14]. Both write on the selection surface. The fill slot, read after the hop is chosen, stays the human guideline shipped with the catalog. DRAFT released a RestBench rewrite; that dump is the documentation baseline in the tables.

Trace-Free+ learns a description generator and evaluates it on RestBench TMDb and Spotify as out-of-domain tools [1]. The generator emits one description per tool. Their protocol supplies the gold API at each subtask and executes that API, so a later step never has to bind a path id after the agent itself chose the hop. DFSDT does not run that way: `choose_tool` reads u_t , `choose_parameter` reads d_t , and the live backend can reject a missing id. A single rewritten description has no fill slot. Copied into hop u_t , extra bind text makes the agent repeat search (Section 4.4); copied into both slots, it is a purpose rewrite in the guideline, which is the DRAFT row. The criterion we score is a typed producer path that is visible only after the hop is chosen, on live HTTP. That is a two-slot write under a released controller, not a teacher-forced one-field rewriter.

Producer–consumer edges outside the agent

AutoRestTest builds a semantic property graph over OpenAPI and uses live responses to fill later test requests [2]. APIPilot treats inferred producer–consumer edges as hypotheses and keeps an edge only after a live injection succeeds [3]. That loop is close to our Bind and Probe steps. Both papers generate tests; they do not write a fill line into DFSDT’s guideline. Arazzo and OpenAPI Links already name a producer path in a workflow spec, for example `$response.body#/id` [5]. The RestBench dump has no such links. The agent we study reads natural-language slots, not an Arazzo runtime.

Wrappers and retrieval fields

Doc2Agent compiles documentation into executable wrappers [4]. The agent then talks to those wrappers, not to the original catalog strings. MFTR standardizes document fields so a retriever can rank tools [11]. Ranking decides which tool page is fetched. The failure in Figure 1 is later: the review hop is already chosen, and the movie id is still unbound. The write we study is a fill line on every hop API and a response line on every search or list API, in the purpose and guideline slots DFSDT reads at those two steps.

3 Method

3.1 Notation and scoring

The input is a finite vendor catalog $\mathcal{T}$. Each tool t is four strings the released DFSDT driver splits across prompts: a purpose u_t (read by `choose_tool`), a guideline d_t (read by `choose_parameter`), a request schema θ_t , and a URL template p_t . Path parameters $\phi(p_t)$ are the placeholders in that template, such as `movie_id` in `/movie/{movie_id}/reviews`. Closed enumerations (`media_type`, `time_window`) are not treated as identifiers.

An agent π maps a user query q and a document set D to a call sequence $a_{1:m}$. RestBench ships, for each query, a gold list $g_{1:k}$ of API names. The ordered-path score counts that list as a correct path if and only if the names appear in order as a subsequence of $a_{1:m}$. The score does not look at arguments. Execution-valid CP adds a second check: no call on that path was rejected by the live backend because a path parameter was missing or invented. A cell that still carries a harness error after the driver’s own resample is dropped from n , not scored as a miss.

Let E be the closed set of enumeration parameters $\{\text{media_type}, \text{time_window}, \text{type}\}$. Tool t is a **hop** if and only if $\phi(p_t) \setminus E \neq \emptyset$. Otherwise t is a **catalog** tool: a search or list endpoint whose URL has no path identifier. The compiler (Algorithm 1) reads $\mathcal{T}$ and writes a new document set D^* . It does

not read gold paths or evaluation traces. Figure 2 states the four steps; Figure 3 shows the two DFSDT prompts after the write.

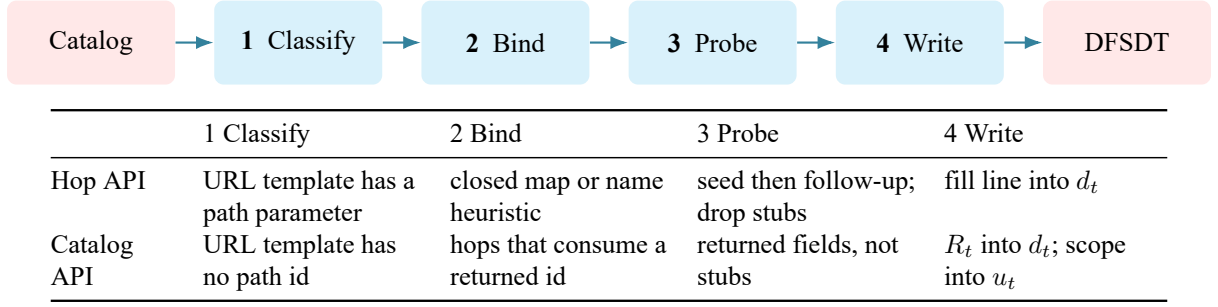


Figure 2: Four compiler steps and what they write on a hop API versus a catalog API. The request schema is unchanged.

3.2 Compiler

Each t enters Algorithm 1 as the vendor Initial dump $(u_t^0, d_t^0, \theta_t^0, p_t)$. Classify reads $\phi(p_t) \setminus E$. A hop receives a fill contract $F_t = \{(\alpha, S_\alpha, \ell_\alpha)\}_{\alpha \in \phi(p_t) \setminus E}$. A catalog tool, and a hop that itself emits further identifiers, receives a response contract $R_t = (\kappa_t, \Delta_t)$: the keys a list payload actually contains, and the hop APIs those identifiers feed. The write is

$$d_t^* = d_t^0 \oplus F_t \oplus R_t, \quad (1)$$

$$u_t^* = u_t^0 \oplus \text{scope}(R_t) \quad \text{on catalog tools only,} \quad (2)$$

$$\theta_t^* = \theta_t^0. \quad (3)$$

$\oplus$ concatenates onto the vendor sentence. Fill is not written into hop u_t , so `choose_tool` still sees the vendor purpose on those APIs. $\text{scope}(R_t)$ is the compression in Section 3.3. On Spotify search, $\text{scope}(R_t)$ is omitted.

Algorithm 1 Compile fill and response contracts from a vendor catalog.

Require: vendor catalog $\mathcal{T}$; optional probe client

Ensure: documents D^*

```

1: for each  $t \in \mathcal{T}$  do
2:    $u_t^* \leftarrow u_t^0$ ;  $d_t^* \leftarrow d_t^0$ ;  $\theta_t^* \leftarrow \theta_t^0$ 
3:   if  $t$  is a hop then
4:     for each  $\alpha \in \phi(p_t) \setminus E$  do
5:       if a closed producer map defines  $\alpha$  then
6:          $S_\alpha, \ell_\alpha \leftarrow$  that map and its JSON path
7:       else
8:          $S_\alpha \leftarrow$  empty- $\phi$  tools whose path is not /me
9:         restrict  $S_\alpha$  to names matching search or the entity of  $p_t$ 
10:         $\ell_\alpha \leftarrow \text{results}[i].\text{id}$  or the entity list path
11:      end if
12:    end for
13:    append  $\alpha$  from  $S_\alpha(\ell_\alpha)$  to  $d_t^*$ 
14:    if a seed or follow-up probe hits some  $\ell_\alpha$  then
15:      append Example: and the observed token to  $d_t^*$ 
16:    end if
17:  end if
18:  if  $t$  is a catalog tool or  $t$  emits further ids then
19:    append  $R_t = (\kappa_t, \Delta_t)$  to  $d_t^*$ 
20:    if  $t$  is a catalog tool and sibling scope is enabled then
21:       $u_t^* \leftarrow u_t^0 \oplus \text{scope}(R_t)$ 
22:    end if
23:  end if
24: end for
25: return  $D^* = \{(u_t^*, d_t^*, \theta_t^*, p_t)\}_{t \in \mathcal{T}}$ 

```

Bind fills S_α in two ways. On TMDB the assignment is a closed map over the 54-API dump. Search, discover, and list endpoints produce `movie_id`; movie and TV detail hops also produce `company_id`, `network_id`, and `season_number`. The map is not restricted to empty- ϕ tools. Thirty-three hops receive at least one triple. On Spotify there is no such map. A tool is treated as a producer of α when its URL has no path identifier, its path is not /me, and either its name contains `search` or it shares the entity token of p_t (13 hops). The JSON path ℓ_α is a second closed table on TMDB (`results[i].id` on a search list; `production_companies[i].id` on a detail hop) and a URL-derived fallback on Spotify.

Bind names the producer and the JSON path. It does not look at a live payload. Probe is an optional live check of that path, used only while compiling D^* .

The client issues five fixed catalog searches (movie, TV, person, company, collection) with five fixed titles. Those titles are not RestBench questions and are not taken from gold paths. The searches return identifiers. Four follow-up hops (credits, reviews, TV detail, season list) are then called with those identifiers, so nested tokens such as `credit_id` and `review_id` can be observed as well.

If a hop’s path parameter α was observed at ℓ_α , Write appends `Example:` $\alpha=v$ to that hop’s fill line. v is a token of the right shape, copied into the frozen guideline so `choose_parameter` can see what a well-formed id looks like. It is not read back at evaluation: RestBench queries are a different set, and the controller is not given the probe log. If the client is absent, the fill line is still written and the `Example:` clause is omitted. Which keys belong in κ_t , including stubs such as `known_for`, is the closed field table in Section 3.3, not a scan of these payloads.

3.3 Contracts

Fill. The fill line names each path parameter, the producer APIs, and the JSON path:

$$\alpha \text{ from } S_\alpha (\ell_\alpha).$$

A hop whose URL is `/movie/{movie_id}/reviews` therefore receives a guideline clause naming `movie_id`, the search and discover producers, and `results[i].id`. The request schema still lists `movie_id` as a string field; the compiler does not add it as a new parameter. The filled slot is the right box of Figure 3.

Response. The response line lists keys the payload actually contains and the hops those identifiers feed:

$$\text{Response: } \kappa_t. \text{ Downstream: } \Delta_t.$$

Δ_t is the set of hop APIs whose URL placeholder is an identifier that t produces. Keys that the catalog names but a probe finds empty (list stubs such as `known_for` on a person search) are marked as stubs, not as sources of an id. Full strings for one hop API and one catalog API are in Appendix A.

Scope. $\text{scope}(R_t)$ is a compressed copy of the same two sentences, written only into catalog u_t : `Response:` becomes `Returns`, and `Downstream:` becomes `not`, so `choose_tool` sees `Returns ...not GET_...`. The guideline keeps the positive downstream list. That compression is enabled on TMDB and disabled on Spotify search. The left box of Figure 3 is the TMDB case.

3.4 Placement

Each DFSDT step is emitted as `Thought: / API Name: / Parameters: [7]`. `Released Inference_DFSDT.py` fills those three fields from three prompts. `task_decompose` sees the purpose taken from the query file plus the variant guideline and writes a plan. `choose_tool` sees only u_t and writes `API Name`. `choose_parameter` sees d_t and θ_t and writes `Parameters`. Figure 3 shows the two document slots and that emit. `Thought` is unconstrained model text, as in ReAct [13]; it is not part of D^* .

choose_tool reads u_t Function: GET_search_person Description: {vendor purpose} Returns results[i].{id, name} not GET_person..._credits, ...	choose_parameter reads d_t Function: GET_movie_movie_id_reviews Guideline: movie_id from GET_search_movie, ... (results[i].id) Parameters: {vendor schema θ_t }
controller emit Thought: {model} API Name: {from choose_tool, reads u_t } Parameters: {from choose_parameter, reads d_t, θ_t }	

Figure 3: Document slots and the controller emit they feed. Braces mark text the compiler does not rewrite. On a hop API, u_t stays the vendor purpose. On a catalog API the purpose is concatenated with $\text{scope}(R_t)$. The schema is the same object in every arm.

Fill is written only into d_t . Figure 4 is the claim about that write: the controller still chooses the next hop; the contract only names how a path parameter is taken from a previous response. The same producer path, placed in hop u_t or lifted into θ_t with a shared “do not invent ids” banner, made the agent repeat search and skip the id hop (Section 4.4). Response scope is written into u_t only on catalog tools.

D^* is built from the vendor Initial dump. The frozen TMDB payload is fill, response, and $\text{scope}(R_t)$. The frozen Spotify payload is fill plus a guideline response without sibling negation.

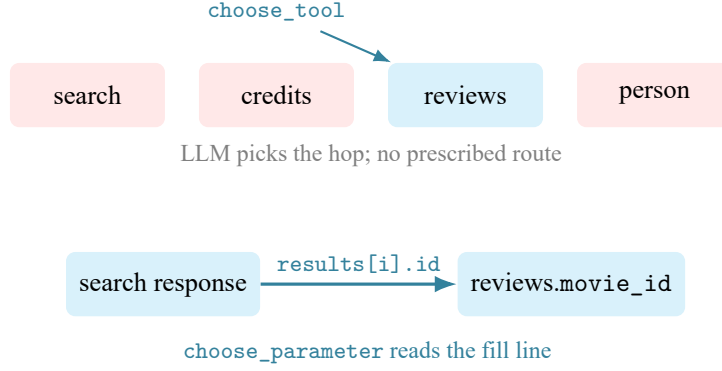


Figure 4: The fill contract names a transition, not a route. `choose_tool` still selects among catalog purposes. Once a hop is chosen, the guideline names the producer path of its path parameter and does not tell the model which sequence of APIs to call.

4 Experiments

4.1 Experiment setup

Controller. Released DFSDT. $T=0.2$, $\text{top_p}=1$, $\text{max_tokens}=2000$, $\text{retrieval_num}=5$.

Documents. DFSDT uses the vendor Initial catalog. DRAFT uses the released rewrite. Ours is the frozen compiled payload above (TMDB) and fill plus guideline response without sibling negation (Spotify).

Data and backends. RestBench-TMDB, $n=100$, live TMDB v3. RestBench-Spotify, $n=57$, live Spotify Web API, same controller. Gold paths are unchanged. Most Spotify gold paths start at user-account endpoints; the frozen map does not treat those calls as producers, so that split is not an execution contrast.

Metrics. Primary: execution-valid CP on finished TMDB cells. Secondary: ordered-path CP, the metric of RestGPT and DRAFT.

Models and seeds. Ling-3.0-Flash and DeepSeek V4, TMDB seeds 1–9. Query-level labels flip across seeds, so intervals are seed-level Student- t 95% ($t_8=2.306$).

4.2 Executable tool use

On every Ling and DeepSeek V4 TMDB seed, Ours is above DRAFT on execution-valid CP (about +42 and +39 pp). Ling also beats the catalog (+6.0 pp); the query-level interval does not cover zero. DeepSeek V4 is on par with the catalog; DRAFT fails on both (15.3% and 23.8%). Per-seed values are in Appendix B.

Table 1: Execution-valid CP (%) on RestBench-TMDB. Seed-mean over 9 seeds, Student-*t* 95%. Bold is the best document on that model.

Model	Initial	DRAFT	Ours
Ling-3.0-Flash (9 seeds)	51.0 [47.2, 54.8]	15.3 [12.7, 18.0]	57.0 [53.9, 60.2]
DeepSeek V4 (9 seeds)	63.1 [60.1, 66.2]	23.8 [21.5, 26.1]	62.2 [57.8, 66.6]

Table 2: Paired seed-mean Δ in execution-valid CP, percentage points, TMDB.

Contrast	Ling (9 seeds)	DeepSeek V4 (9 seeds)
Ours – DRAFT	+41.6 [+36.6, +46.5]	+38.4*
Ours – Initial	+6.0 [+0.2, +11.8]	−0.9 [−6.5, +4.7]

*DeepSeek V4 vs DRAFT: every seed is positive; seed-mean gap +38.7 on triple-shared finished cells. Seed intervals are decode noise. A paired bootstrap over the 100 queries, seeds averaged inside a query: Ling Ours–Initial +6.0 [+2.1, +10.0]; DeepSeek V4 −0.9 [−4.8, +3.0].

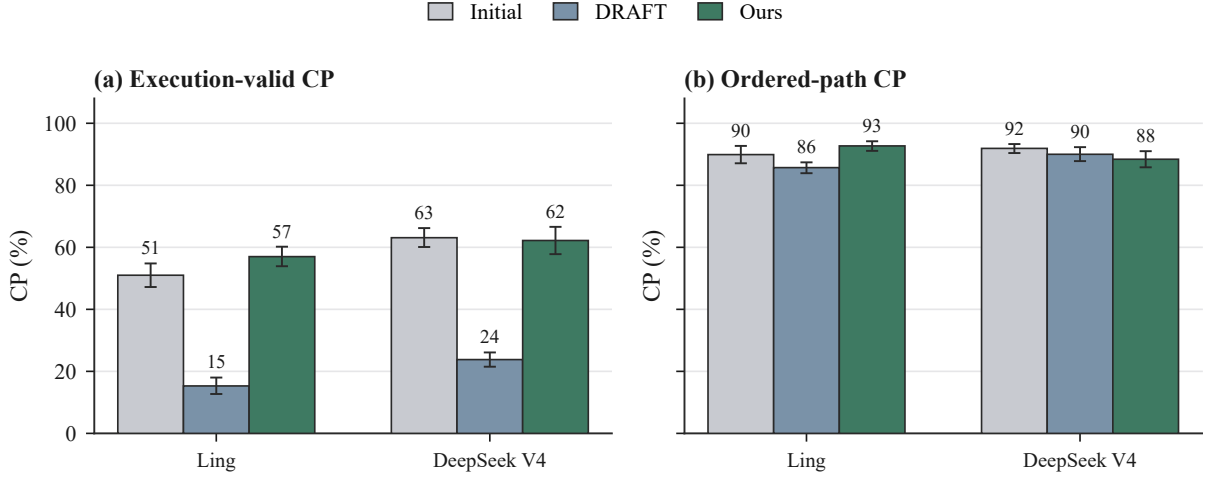


Figure 5: RestBench-TMDB, same runs as Tables 1 and 3. Seed-mean over 9 seeds, Student-*t* 95% whiskers. (a) Primary metric. (b) Ordered-path CP.

DRAFT’s descriptions look helpful and fill badly. Ordered-path CP can stay in the mid-80s to mid-90s while three-quarters of those traces fail HTTP (Table 3). DeepSeek V4 already executes well on the original catalog (63%). Ours stays on par; DRAFT does not.

4.3 Sibling negation on DeepSeek V4

On DeepSeek V4, Ours misses more first producers (35 vs 25 of 900) and more final consumers (49 vs 30) than Initial. The typical miss is not a wrong hop. Decompose has already named the gold consumer; choose_tool then returns no valid choice because the catalog document contains not GET_<that consumer>. Ling, on the same string, reads the negation as “this search is not the credits call” and still selects credits later. The fill line in the guideline is unchanged. That is why DeepSeek V4 stays on par with the catalog on execution-valid CP while ordered-path CP falls: the traces that survive selection still fill. Spotify search was compiled without sibling not GET_ names.

4.4 Placement

On TMDB Ling, one seed, putting path-parameter text into the schema plus a shared “do not invent ids” banner scored 80 CP with 10 first-API stuck queries. A hop gate in choose_tool scored 82.

Table 3: Seed-mean ordered-path CP (%) on RestBench-TMDB. Same runs as Table 1.

Model	Initial	DRAFT	Ours
Ling-3.0-Flash	89.9 [87.1, 92.7]	85.7 [83.9, 87.4]	92.7 [91.1, 94.2]
DeepSeek V4	91.9 [90.4, 93.3]	90.0 [87.8, 92.3]	88.4 [85.8, 91.0]

Fill in the guideline only scored 93 with 2 stuck. Extra text in u_t of hop APIs makes the agent repeat search. Response scope was therefore restricted to catalog u_t . Replacing the purpose with structural fingerprints collapses both models (Ling 36/48, DeepSeek V4 46/48 on a frozen 24-query panel). Those two measurements are why fill stays in d_t and why response scope is not written into hop u_t .

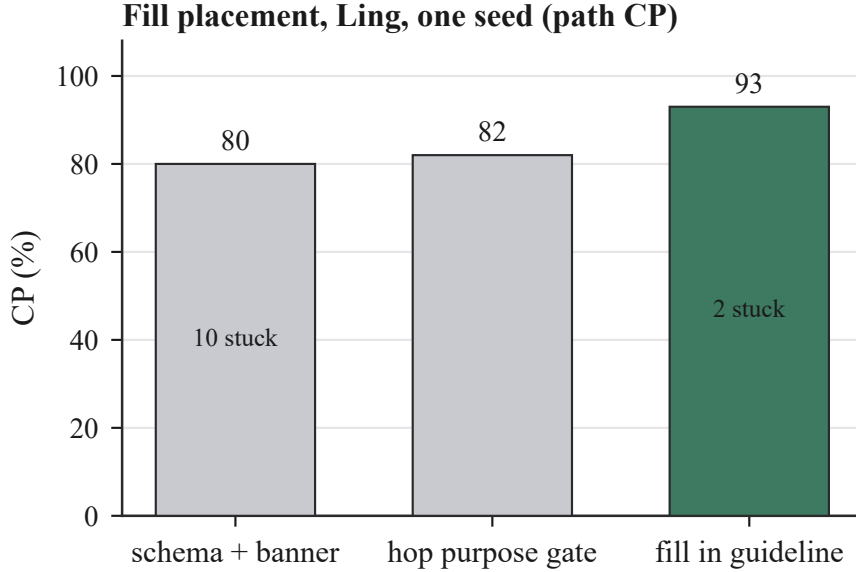


Figure 6: Where the fill line is written. Path CP, TMDB Ling, one seed. Grey bars are the two rejected slots; the green bar is the frozen write. Stuck counts are first-API repeats, reported only where that seed was counted.

5 Limitations

The measured gain is a documentation effect inside the released DFSDT controller: the search procedure, the request schema, and the models stay fixed. The Spotify compiler does not bind user-account hops, so that split is not an execution contrast. Sibling negation on the TMDB catalog selection surface helps Ling and blocks DeepSeek V4; that is a limit of the same payload, not a second result. Ling and DeepSeek V4 TMDB runs used unpinned provider routing.

6 Future work

We will evaluate the same three documentation arms on ToolBench G3 instructions whose later required identifier is absent from the query and must be taken from an earlier response (Appendix C). That setting asks the fill line to transform a payload, which RestBench-TMDB does not test. A third model family will be reported as a completed seed panel.

7 Conclusion

Vendor REST catalogs name the next endpoint and leave implicit how a later call should be filled from an earlier response. A free-form purpose rewrite can keep gold API names while the live call still fails. The intervention we can defend is a slot write inside DFSDT: a fill contract and a response contract in the guideline, hop purpose left as the vendor wrote it. That raises execution-valid CP on Ling relative to both the catalog and that rewrite. On DeepSeek V4 the same write matches the catalog and the rewrite fails. The fill line is the part that transfers; compressing sibling hops into not GET_ on the selection surface does not transfer across families.

References

- [1] Ruocheng Guo, Kaiwen Dong, Xiang Gao, and Kamalika Das. Learning to rewrite tool descriptions for reliable LLM-agent tool use. *arXiv preprint arXiv:2602.20426*, 2026.
- [2] Myeongsoo Kim, Tyler Stennett, Dhruv Shah, Saurabh Sinha, and Alessandro Orso. A multi-agent approach for REST API testing with semantic graphs and LLM-guided test generation. In *ICSE*, pages 1409–1421, 2025.
- [3] Tu Nguyen, Thanh Nguyen, Huy Nguyen, Viet Nguyen, Tien N. Nguyen, and Vu Nguyen. REST API testing with verified LLM-inferred dependencies and response-driven refinement. *arXiv preprint arXiv:2608.17546*, 2026.
- [4] Xinyi Ni, Haonan Jian, Qiuyang Wang, Vedanshi Chetan Shah, and Pengyu Hong. Doc2Agent: Scalable generation of tool-using agents from API documentation. *arXiv preprint arXiv:2506.19998*, 2025.
- [5] OpenAPI Initiative. Arazzo specification. <https://www.openapis.org/arazzo-specification>, 2024.
- [6] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2024.
- [7] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *ICLR*, 2024.
- [8] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Ji, Shuaiqiang Cai, Yuning Wu, Weiwen Bi, Haofen Yu, Junjie Chen, and Hua Wu. From exploration to mastery: Enabling LLMs to master tools via self-driven interactions. In *ICLR*, 2025.
- [9] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.
- [10] Yifan Song, Weimin Xiong, Dawei Zhu, Cheng Li, Ke Wang, Ye Tian, and Sujian Li. Rest-GPT: Connecting large language models with real-world RESTful APIs. In *arXiv preprint arXiv:2306.06624*, 2023.
- [11] Yichen Tang, Weihang Su, Yiqun Liu, and Qingyao Ai. Multi-field tool retrieval. *arXiv preprint arXiv:2602.05366*, 2026.
- [12] Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. ToolGen: Unified tool retrieval and calling via generation. In *ICLR*, 2025.

- [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- [14] Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Kan Ren, Dongsheng Li, and Deqing Yang. EASYTOOL: Enhancing LLM-based agents with concise tool instruction. In *NAACL*, 2025.

A Worked examples

DRAFT prints a full documentation rewrite in its appendix; RestGPT prints gold paths next to the user question. We do the same for two RestBench-TMDB queries: the strings below are the frozen dumps, not a paraphrase.

A.1 Search then credits

Query 1: “Who was the lead actor in the movie The Dark Knight?” Gold path: GET_search_movie, GET_movie_movie_id_credits. The second URL is /movie/{movie_id}/credits. Table 4 is what each arm writes into the guideline d_t of that hop. choose_tool still sees the vendor purpose on the hop (“Get the cast and crew for a movie.”). The request schema is unchanged.

Table 4: Guideline d_t on GET_movie_movie_id_credits for query 1. DRAFT copies the same paragraph into u_t .

Arm	d_t (hop)
Initial	Get the cast and crew for a movie.
DRAFT	Prose rewrite of the same purpose. It says the movie ID is in the URL. It does not name a producer call or a JSON path.
Ours	Get the cast and crew for a movie. movie_id from GET_search_movie, GET_discover_movie, GET_movie_popular, ... (results[i].id). Response: cast[i].credit_id or crew[i].credit_id. Downstream: GET_credit_credit_id.

The catalog search entry is one sentence (“Search for movies.”). Ours appends a response line to d_t and a compressed scope line to u_t :

```

 $d_t$ : Response: results[i].{id,title,release_date,overview}.
Downstream: GET_movie_movie_id_credits, GET_movie_movie_id_images, ...
 $u_t$ : Search for movies. Returns results[i].{id,title,release_date,overview}.
not GET_movie_movie_id_credits, GET_movie_movie_id_images, ...

```

A.2 Collection then review

Query 38 is Figure 1: “Give me a review of a movie from the collection The Fast and the Furious.” Gold path: GET_search_collection, GET_collection_collection_id, GET_movie_movie_id_reviews. The last hop still needs a movie_id. Collection detail is the producer of that id (parts[i].id); the reviews fill line still names search and list APIs, as in the closed map (Appendix C).

Table 5: Frozen strings on the two hops of query 38. Vendor changelog text on the collection page is omitted.

Hop	Slot	Written string
search collection	u_t Ours	Search for collections. Returns results[i].{id,name}. not GET_collection_collection_id_images, GET_collection_collection_id.
search collection	d_t Ours	Search for collections. Response: results[i].{id,name}. Downstream: GET_collection_collection_id_images, GET_collection_collection_id.
collection by id	d_t Ours	collection_id from GET_search_collection (results[i].id). Response: id, name, parts[i].id, production_companies.
movie reviews	d_t Initial	Get the user reviews for a movie.
movie reviews	d_t DRAFT	“This API retrieves user reviews for a specific movie using the movie ID included in the URL path as a required parameter ...” Copied into u_t . An attached example uses movie ID 11111.
movie reviews	d_t Ours	movie_id from GET_search_movie, GET_discover_movie, GET_movie_popular, ... (results[i].id). Response: results[i].id. Downstream: GET_review_review_id.

Under Initial the collection id can be filled while movie_id stays a stub; HTTP rejects the review call. Under DRAFT both stubs stay empty. Under Ours the controller reads parts[i].id from the collection payload and the review call returns a user review.

B Per-seed TMDB results

Tables 1 and 2 report seed means. Tables 6 and 7 are the nine TMDB seeds that enter those means. A leftover driver or transport error is a miss and stays in the denominator. Seeds are 1–9 ($T=0.2$). No seed is dropped.

Table 6: Execution-valid CP (%) on RestBench-TMDB, Ling-3.0-Flash, one seed per row.

Seed	Initial	DRAFT	Ours
1	50.0	14.0	58.0
2	47.0	10.0	65.0
3	50.0	17.0	62.0
4	55.0	21.0	54.0
5	53.0	18.0	54.0
6	57.0	16.0	54.0
7	41.0	11.0	56.0
8	50.0	14.0	53.0
9	56.0	17.0	57.0
Mean	51.0	15.3	57.0

Table 7: Execution-valid CP (%) on RestBench-TMDB, DeepSeek V4, one seed per row.

Seed	Initial	DRAFT	Ours
1	63.0	24.0	53.0
2	55.0	23.0	64.0
3	63.0	22.0	67.0
4	66.0	20.0	58.0
5	67.0	29.0	59.0
6	59.0	24.0	61.0
7	67.0	27.0	59.0
8	65.0	25.0	68.0
9	63.0	20.0	71.0
Mean	63.1	23.8	62.2

Ours is above DRAFT on every seed of both families. Against Initial, Ling is above on seven of nine seeds and DeepSeek V4 on five of nine. That is the decode-level spread behind the seed-mean intervals, not a second experiment.

C Artifacts and protocol

C.1 Bind versus gold paths

Bind is a closed map over the 54-API TMDB dump. It does not read RestBench gold paths. Table 8 is a post-hoc audit: for each gold hop that is not the first call, whether some closed-map producer already appears earlier on that gold path.

Table 8: Closed-map producers on RestBench-TMDB gold paths. 122 gold hops that are not the first call.

Gold hop (not first)	Count
At least one closed-map producer earlier on the path	91
Needs an intermediate hop as the producer	31

The 31 residual hops take `person_id` from movie credits, `movie_id` from a person’s credit list, or `season_number` from a TV object. The fill line still names search and list APIs. The overlap is an audit, not a training signal.

C.2 DRAFT slots

The released TMDB DRAFT dump has one `description` field and no separate `tool_description`. The DFSDT adapter copies that field into both u_t and d_t on all 54 tools. Every hop also carries an `example`; seven of those examples use the invented identifiers 11111 or 13579. Table 5 quotes one. DRAFT’s 15.3% execution-valid CP is not an empty guideline. It is a purpose rewrite, with fake identifiers, in both slots.

C.3 Named-entity identifiers

Execution-valid CP accepts any HTTP-valid path id. A second check freezes named-entity ids once (Titanic, Sofia Coppola, Star Wars collection parts) and, on popular and trending gold paths, harvests ids from the producer of that trace.

Table 9: Named-id CP (%), same TMDB traces as Table 1. Query-level Ours—Initial on Ling is +2.0 $[-0.9, +4.9]$ and covers zero.

Model	Initial	DRAFT	Ours
Ling-3.0-Flash	40.2	12.0	42.2
DeepSeek V4	50.2	20.3	48.9

The fill line repairs stubs against DRAFT. On this freeze it does not beat the catalog at picking the named title.

C.4 Probe seeds

The frozen TMDB payload records `examples_from_probe=0` and empty `seed_ids`. No probe token was written into d_t . The five titles reserved for a probe, had one run, were Inception, Game of Thrones, Tom Hanks, Disney, and Star Wars. None of those strings occur in `TMDB_ours.json`. Query 38 uses The Fast and the Furious; that title is not a seed.

C.5 ToolBench G3 nested slice

Released G3 has 100 instructions. Table 10 splits them by whether a later required identifier is already in the user text. Indices: `artifacts/documentation/G3_nested.json`.

Table 10: Released ToolBench G3 instructions, $n=100$.

Slice	Count
Nested: later required id absent from the query	40
Independent: every gold identifier already in the query	39
Underspecified: an identifier is missing and no earlier gold API	21

C.6 Reproducibility

Contracts are compiled by `src/tooldoc_nir/restbench_ours.py`. The released DFSDT controller is driven by `python -m tooldoc_nir.restbench_table`. Decoding is $T=0.2$, `top_p=1`, `max_tokens=2000`, `retrieval_num=5`. Traces live under `artifacts/results/`. Vendor keys stay in a local `.env`.